MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

24 de fevereiro de 2026

1 Informações preliminares

2 Introdução

3 Brevíssima revisão de MC358

4 Perguntas ou comentários?

Informações preliminares

Agradecimentos

Vários exemplos, imagens e conteúdos utilizados nesses slides me foram gentilmente cedidos pelo professor *Lehilton Lelis Chaves Pedrosa*, que, por sua vez, usou bastante material criado ou coletado pelo prof. *Cid Carvalho de Souza*, pela profa. *Cândida Nunes da Silva* (particularmente com modificações do prof. *Orlando Lee*) e pelo prof. *Flávio Keidi Miyazawa*.

Apresentação do professor

- USP, Unicamp, Luxembourg, KU Leuven
- Professor do Instituto de Computação
- Departamento (área de concentração): Teoria da Computação
- Especialidade: Criptografia

Pré-requisitos do curso

Basicamente, MC358

- Ler (e entender) provas matemáticas
- Escrever suas próprias provas matemáticas
- Indução, recorrências, funções...

Pré-requisitos do curso

Basicamente, MC358

- Ler (e entender) provas matemáticas
- Escrever suas próprias provas matemáticas
- Indução, recorrências, funções...

Teremos uma brevíssima revisão

Objetivos do curso

- Demonstrar correção (corretude) de um algoritmo
  - como ter certeza de que a saída está correta
  - como convencer outras pessoas disso

Objetivos do curso

- Demonstrar correção (corretude) de um algoritmo
  - como ter certeza de que a saída está correta
  - como convencer outras pessoas disso
- Analisar a complexidade de um algoritmo
  - como estimar a quantidade de recursos utilizados
  - recursos podem ser tempo, memória, acesso a rede etc.

Objetivos do curso

- Demonstrar correção (corretude) de um algoritmo
  - como ter certeza de que a saída está correta
  - como convencer outras pessoas disso
- Analisar a complexidade de um algoritmo
  - como estimar a quantidade de recursos utilizados
  - recursos podem ser tempo, memória, acesso a rede etc.
- Utilizar técnicas conhecidas de projeto de algoritmos
  - divisão e conquista, programação dinâmica etc.
  - utilizar recursão adequadamente

Informações práticas

Sobre os slides: são apenas para guiar os estudos, são praticamente só uma lista dos tópicos abordados nas aulas. **Não** devem ser usados para estudo! Para estudar e praticar, **utilize o livro-texto** indicado.

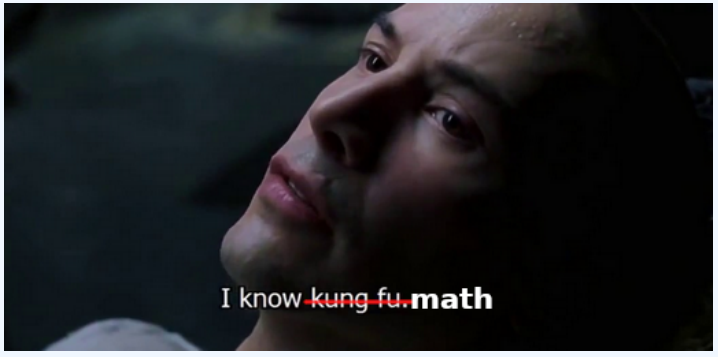
Informações práticas

Sobre os slides: são apenas para guiar os estudos, são praticamente só uma lista dos tópicos abordados nas aulas. **Não** devem ser usados para estudo! Para estudar e praticar, **utilize o livro-texto** indicado.

- Página do curso: Classroom
- PDD: Classroom
- Avaliações: 2 provas, 3 ou 4 testes, 2 ou 3 listas de exercício, 2 projetos de programação.
  - Listas corrigidas parcialmente
- Monitorias: qual horário?

Aprendizado é uma tarefa ativa!

Não estamos na Matrix! O professor não consegue inserir o conhecimento na sua cabeça.



Faça exercícios, revise as aulas, leia a bibliografia recomendada...

Introdução

Problemas computacionais

Um problema computacional é uma relação entre um conjunto de instâncias e um conjunto de soluções:

- uma instância é um conjunto de valores conhecidos
- uma solução é um conjunto de valores a computar
- cada instância corresponde a uma ou mais soluções

Problemas computacionais

Um problema computacional é uma relação entre um conjunto de instâncias e um conjunto de soluções:

- uma instância é um conjunto de valores conhecidos
- uma solução é um conjunto de valores a computar
- cada instância corresponde a uma ou mais soluções

$$\mathcal{C} = \{\text{instâncias}\},\ \mathcal{S} = \{\text{soluções}\} \Rightarrow \text{problema } \mathcal{P} \subset \mathcal{C} \times \mathcal{S}$$

Exemplo de problema: teste de primalidade

Problema: determinar se um dado número é primo

- instâncias: números inteiros, i.e., $\mathbb{Z}$
- soluções: sim ou não

Exemplo:

- Instância: 9411461
- Solução: sim

Exemplo:

- Instância: 8411461
- Solução: não

Exemplo de problema: ordenação

Problema: ordenar os elementos de um vetor

- instâncias: conjunto de vetores de inteiros
- soluções: conjunto de vetores de inteiros em ordem crescente

Exemplo:

- Instância:

1										n
33	55	33	44	33	22	11	99	22	55	77

- Solução:

1										n
11	22	22	33	33	33	44	55	55	77	99

Exemplo de problema: multiplicação polinomial

Problema: multiplicar dois polinômios

- instâncias: ???
- soluções: ???

Exemplo de problema: multiplicação polinomial

Problema: multiplicar dois polinômios

- instâncias: ???
- soluções: ???

Exemplo:

- Instância: $(2X + 3X^5,\ X + 1)$
- Solução: $3X^6 + 3X^5 + 2X^2 + 2X$

Algoritmos

Um algoritmo é uma sequência de instruções que

- recebe uma instância de um problema computacional
- devolve uma solução correspondente à instância recebida

Algoritmos

Um algoritmo é uma sequência de instruções que

- recebe uma instância de um problema computacional
- devolve uma solução correspondente à instância recebida

Observações:

- a instância recebida é chamada de entrada
- a solução devolvida é chamada de saída
- toda instrução deve ser bem definida

Usaremos pseudocódigo:

```
Add(v, n)
Input: vetor v[0, ..., n − 1] ∈ ℤ^n, n ≥ 1
Output: a soma das entradas ímpares de v
s ← 0
for (i ← 0; i < n; i++) do
    if v[i] mod 2 = 1 then
        s ← s + v[i]
return s
```

Más práticas

- Não escreva frases confusas:

```
1 if A[i] > A[j] then
2   troque as posições dos elementos
```

ou

```
1 chave ← pega o próximo elemento
2 i ← procura posição da chave
```

- Não escreva frases confusas:

```
1 if A[i] > A[j] then
2     troque as posições dos elementos
```

ou

```
1 chave ← pega o próximo elemento
2 i ← procura posição da chave
```

- Evite algoritmos complicados ou com muitas variáveis. Prefira subalgoritmos.

Algoritmos bons e ruins

Não nos preocupamos com a "velocidade" dos computadores que vão executar os algoritmos, mas com os algoritmos em si.

Na maioria das vezes, vamos avaliar quão bom um algoritmo é contando o número de operações básicas que ele executa em função do tamanho da entrada.

Por exemplo, considere algoritmos que ordenam vetores com n elementos:

- Algoritmo $\mathcal{A}$ executa $2n^2$ operações
- Algoritmo $\mathcal{B}$ executa $50n \log n$ operações

Considere dois computadores:

- $C_{\mathcal{A}}$ executa $\mathcal{A}$ com processador de 1GHz
- $C_{\mathcal{B}}$ executa $\mathcal{B}$ com processador de 10MHz (100 vezes mais lento)

Qual máquina ordena um milhão de elementos primeiro?

1. $C_{\mathcal{A}}$ leva
$$\frac{2 \cdot (10^6)^2 \text{instruções}}{10^9 \text{instruções /segundo}} \approx 2000 \text{ segundos}$$
2. $C_{\mathcal{B}}$ leva
$$\frac{50 \cdot (10^6 \log_2 10^6) \text{instruções}}{10^7 \text{instruções /segundo}} \approx 100 \text{ segundos}$$

- A máquina $C_{\mathcal{B}}$ foi vinte vezes mais rápida do que $C_{\mathcal{A}}$

Qual máquina ordena um milhão de elementos primeiro?

1. $C_{\mathcal{A}}$ leva
$$\frac{2 \cdot (10^6)^2 \text{instruções}}{10^9 \text{instruções /segundo}} \approx 2000 \text{ segundos}$$
2. $C_{\mathcal{B}}$ leva
$$\frac{50 \cdot (10^6 \log_2 10^6) \text{instruções}}{10^7 \text{instruções /segundo}} \approx 100 \text{ segundos}$$

- A máquina $C_{\mathcal{B}}$ foi vinte vezes mais rápida do que $C_{\mathcal{A}}$
- Se fossem 10 milhões de elementos, $C_{\mathcal{A}}$ levaria 2,3 dias e $C_{\mathcal{B}}$ 20 minutos

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$
n^3	N_3	$4{,}64N_3$	$10N_3$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$
n^3	N_3	$4{,}64N_3$	$10N_3$
n^5	N_4	$2{,}5N_4$	$3{,}98N_4$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$
n^3	N_3	$4{,}64N_3$	$10N_3$
n^5	N_4	$2{,}5N_4$	$3{,}98N_4$
2^n	N_5	$N_5 + 6{,}64$	$N_5 + 9{,}97$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$
n^3	N_3	$4{,}64N_3$	$10N_3$
n^5	N_4	$2{,}5N_4$	$3{,}98N_4$
2^n	N_5	$N_5 + 6{,}64$	$N_5 + 9{,}97$
3^n	N_6	$N_6 + 4{,}19$	$N_6 + 6{,}29$

E se usarmos um supercomputador? Qual o tamanho máximo da entrada que conseguiremos processar no mesmo tempo (e.g., 1 dia)?

$f(n)$	Computador atual	$100\times$mais rápido	$1000\times$mais rápido
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31{,}6N_2$
n^3	N_3	$4{,}64N_3$	$10N_3$
n^5	N_4	$2{,}5N_4$	$3{,}98N_4$
2^n	N_5	$N_5 + 6{,}64$	$N_5 + 9{,}97$
3^n	N_6	$N_6 + 4{,}19$	$N_6 + 6{,}29$

Se só tivermos um algoritmo ruim, não iremos resolver problemas muito maiores.

Conclusões

Algumas conclusões:

- Projetar algoritmos melhores pode levar a ganhos extraordinários de desempenho.
- Isso é tão importante quanto o projeto de hardware.
- O ganho obtido ao melhorar a complexidade de um algoritmo não poderia ser obtida simplesmente com o avanço da tecnologia.
- Queremos estudar principalmente algoritmos fundamentais, que produzem avanços em outras componentes básicas das aplicações (pense nos compiladores, buscadores na internet, classificadores, etc).

Brevíssima revisão de MC358

Demonstrações matemáticas

Uma demonstração (prova) é um texto (frequentemente com muitos símbolos matemáticos) que visa convencer o leitor de que uma proposição vale.

Cada passo da demonstração deve ser justificado.

A demonstração vale se cada passo vale, por isso, é fácil de ser *verificada*.

Técnicas de prova

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$

Exemplo:
$X \cup Y \subseteq Y \Rightarrow X \subseteq Y$

Técnicas de prova

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$
- Contrapositiva:
  - Assuma $\neg A$
  - Derive $\neg B$
  - Conclua $A \Rightarrow B$

Técnicas de prova

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$
- Contrapositiva:
  - Assuma $\neg A$
  - Derive $\neg B$
  - Conclua $A \Rightarrow B$

Exemplo:

$n = p \cdot q \Rightarrow p \leq \sqrt{n}$ ou $q \leq \sqrt{n}$

Técnicas de prova

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$
- Contrapositiva:
  - Assuma $\neg A$
  - Derive $\neg B$
  - Conclua $A \Rightarrow B$
- Redução ao absurdo:
  - Assuma $\neg C$
  - Derive *Falso*
  - Conclua C

Técnicas de prova

- Prova direta:
  - Assuma A
  - Derive B
  - Conclua $A \Rightarrow B$
- Contrapositiva:
  - Assuma $\neg A$
  - Derive $\neg B$
  - Conclua $A \Rightarrow B$
- Redução ao absurdo:
  - Assuma $\neg C$
  - Derive *Falso*
  - Conclua C

Exemplo:
$\forall (r, i) \in \mathbb{Q}^* \times \mathbb{I},\ r \cdot i \in \mathbb{I}$

Indução matemática

É uma técnica que nos permite provar que uma propriedade $P(n)$ vale para todo n num subconjunto infinito dos inteiros:

- $\mathcal{S} = \{k \in \mathbb{N} : k \geq n_0\}$
- $P(n)$ algum predicado (uma propriedade)
- Provamos $\forall n \in \mathcal{S}, P(n)$

Estrutura da prova por indução

- Caso base: $P(n_0)$
- Hipótese de indução: $\exists k \in \mathbb{N} : k \geq n_0$ e $P(k)$
- Passo indutivo: $P(k+1)$

Exemplos de indução

- $\forall n \in \mathbb{N}, \sum_{i=0}^{n} i = \frac{n \cdot (n+1)}{2}$

Exemplos de indução

- $\forall n \in \mathbb{N}, \sum_{i=0}^{n} i = \frac{n \cdot (n+1)}{2}$
- $\forall n \in \mathbb{N}, \sum_{i=0}^{n} i^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6}$

Exemplos de indução

- $\forall n \in \mathbb{N}, \sum_{i=0}^{n} i = \frac{n \cdot (n+1)}{2}$
- $\forall n \in \mathbb{N}, \sum_{i=0}^{n} i^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6}$
- (PG) $\forall (r, n) \in \mathbb{R}^* \times \mathbb{N}, \sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1}$

Variações da prova por indução

Incremento no passo indutivo maior que 1:

- Se quiser usar incremento de m
- Caso base: $P(n_0)$, $P(n_0 + 1)$,...,$P(n_0 + m - 1)$
- Passo indutivo: $P(k) \Rightarrow P(k + m)$

Variações da prova por indução

Incremento no passo indutivo maior que 1:

- Se quiser usar incremento de m
- Caso base: $P(n_0)$, $P(n_0+1)$,...,$P(n_0+m-1)$
- Passo indutivo: $P(k) \Rightarrow P(k+m)$

Exemplo:
Mostre que $n^3 - n$ é divisível por 3 para todo natural n.

Perguntas ou comentários?